

Technical Report: Uncovering and Mitigating Linux MGLRU’s Generational Imbalance in Android at Scale

Zicheng Wang

Honor Device Co., Ltd

wangzicheng@honor.com

Ligeng Chen

Honor Device Co., Ltd

chenlg@smail.nju.edu.cn

Yinggang Guo

Honor Device Co., Ltd

guoyinggang@honor.com

Tao Wang

Honor Device Co., Ltd

tao.wangtao@honor.com

Lu Liu

Honor Device Co., Ltd

liulu.liu@honor.com

Fei Zeng

Central South University

feizeng@csu.edu.cn

Hao Wu

Nanjing University

hao.wu@nju.edu.cn

Feng Lyu

Central South University

fenglyu@csu.edu.cn

April 2026

Abstract

The LRU implementation is the core policy engine of the Linux virtual memory subsystem. It determines how physical pages are prioritized, retained, and reclaimed under memory pressure. Multi-Gen LRU (MGLRU) was introduced as its successor, representing a paradigm shift from the long-standing dual-list design. Yet its adoption remains limited: MGLRU improves some workloads but regresses others, and the root causes of these regressions remain unclear. HONOR has deployed MGLRU across its full range of devices with annual shipments exceeding 70 million units. At this scale, we identified a fundamental design issue: MGLRU couples the generation timelines of anonymous and file pages, causing systematic over-eviction of file pages and under-reclamation of anonymous pages under typical Android workloads. We propose a mitigation based on userspace-triggered per-memcg aging. An A/B test on 8 GB commercial devices, spanning ~ 300 users and ~ 30,000 device-hours, shows that the approach improves app keep-alive by 15%, hot start latency by 16%, and IME popup delay by 45%.

1 Introduction

The LRU implementation serves a role in the virtual memory subsystem analogous to what the process scheduler plays for CPU: it is the policy engine that decides how a scarce physical resource, DRAM, is shared among competing demands. Every decision the LRU makes about which pages to retain and which to release directly shapes application latency, background retention, and overall system responsiveness. Getting this policy right is critical on any platform, but especially on mobile devices where memory is tightly constrained.

MGLRU [1] was introduced to replace the decades-old ac-

tive/inactive dual-list design with a multi-generation framework. It is a paradigm shift in how the kernel manages physical memory. In practice, however, the results have been mixed. Some workloads benefit from the finer-grained generational tracking, while others regress. Not all Android OEMs have enabled MGLRU in production, and those who have not cite unpredictable memory behavior as the primary concern. Understanding *why* MGLRU regresses in certain scenarios remains an open question.

HONOR is in a unique position to answer this question. We enabled MGLRU across all commercial devices, covering kernel versions 5.15 through 6.12 and memory sizes from 4 GB to 24 GB, with annual shipments exceeding 70 million units. At this scale, we observed a consistent pattern of MemAvailable loss that could not be explained by workload variation alone.

To quantify the problem, we constructed a sequential launch stress test on a 16 GB device: launching 23 popular apps back-to-back, followed by the camera. As shown in Figure 1, under MGLRU, MemAvailable dropped to approximately 6,060 MB. Under the traditional dual-list LRU, it stayed at about 6,982 MB. The gap is nearly 1 GB. The same gap exists in daily usage, but the stress test makes it easier to reproduce and measure.

We traced the root cause to a fundamental design choice in MGLRU: anonymous and file pages share a single generation timeline. Under Android workloads, this coupling pushes anonymous pages into the youngest generation while scattering file pages across older ones, inverting the reclaim priority that Android expects. The mechanism also bypasses the swappiness parameter, so the standard tuning knob cannot correct the imbalance. This is not specific to Android. Any workload that benefits from preserving file pages, whether on mobile devices or servers, would experience a similar imbalance under the coupled design. We confirmed this with a community-provided server benchmark, which showed the

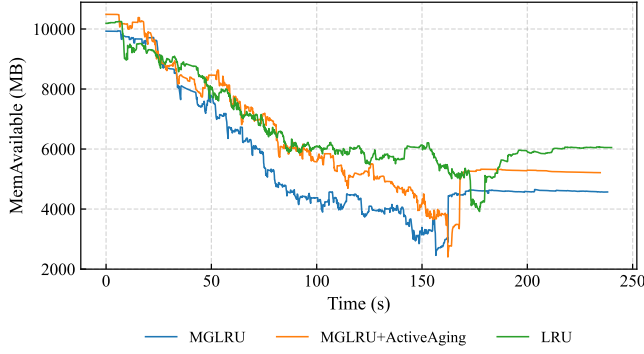


Figure 1: MemAvailable under sequential launch with LRU and MGLRU

same pattern of file page over-eviction.

This paper makes three contributions. First, we present the first large-scale production quantification of MGLRU’s generational imbalance on Android, based on data from over 70 million shipped devices. Second, we identify the root cause at the code level, covering both the page placement rules and the aging gate logic. Third, we propose a userspace memcg aging mitigation and validate it through a commercial A/B test spanning hundreds of users and tens of thousands of device-hours.

2 Background

2.1 Traditional LRU and Its Limitations

The Linux kernel has long used active/inactive dual lists for page reclaim. Recently accessed pages are added to the active list. Pages that have not been accessed for a while get demoted to the inactive list. Reclaim only takes pages from the inactive list. The problem is that this model has only two states. It cannot distinguish between a page accessed once and one accessed hundreds of times in the last hour. Both end up in the active list. Reclaim decisions become coarse-grained as a result.

2.2 MGLRU Design

MGLRU replaces the two-state model with a multi-generation framework. Pages are classified into up to four generations ordered by access recency, providing a graduated temperature scale that the dual-list model cannot express. It maintains two sequence numbers: `max_seq` for the youngest generation and `min_seq` for the oldest. Each page type has its own `min_seq` but both share the same `max_seq`. Reclaim only evicts from the oldest generation.

Entry. When a page enters MGLRU, `lru_gen_add_folio()` assigns its initial generation based on page type and flags: active pages to the youngest, anonymous to the second youngest, default file pages to a

middle generation. This is richer than the traditional LRU, where any first-access page enters the active list directly.

Aging. MGLRU tracks access through two mechanisms. For `mmap`’d pages, the kernel scans page tables and checks the hardware Accessed bit (DBM on ARM). Pages with the bit set are promoted to the youngest generation via `folio_update_gen()`. For file pages accessed via `read()/write()` syscalls, which may lack PTE mappings, `folio_mark_access()` calls `folio_inc_gen()` to bump them one generation up per access. These two paths cover both dominant access patterns, an advantage over the traditional LRU’s single reordering mechanism.

Reclaim and coupling. During isolation from the oldest generation, pages that should not yet be evicted, i.e., dirty, under writeback, or actively referenced, are bumped one generation up instead of being reclaimed. This gives protected pages multiple chances, unlike the traditional model’s all-or-nothing approach. However, there is a critical trade-off: **anonymous and file generations are coupled**. Both share `max_seq`, and when the gap between them reaches 2 or more, the lagging type is forced forward to catch up. As we show in Section 3, this simplification causes problems under certain workloads.

2.3 Android Memory Scenario

Android uses a per-app memcg model. Each application runs in its own memory control group, and a typical device keeps about 20 apps alive simultaneously. An app’s lifecycle is straightforward: launch, foreground (visible, user interaction), background (invisible), and freeze. At launch and during foreground activity switches, the system preloads resources at the GB scale. The foreground phase demands sufficient memory supply to ensure responsiveness. Once an app moves to the background, its memory should be reclaimed appropriately. Throughout this lifecycle, if memory pressure rises, `kswapd` performs async reclaim in the background and the low-memory killer terminates apps as a last resort. Reclaiming efficiency, therefore, directly affects user experience.

There is another important factor: the reclaim cost of anonymous and file pages differ by roughly an order of magnitude. Anonymous pages must be swapped out, which involves allocating a swap slot, inserting into the swap cache, unmapping page table entries, and compressing via zram or writing to a swap device. File pages, on the other hand, are cheap to reclaim. Over 90% of them are clean. Dropping a clean file page from the page cache requires almost no I/O.

Given this asymmetry, Android’s preferred strategy is clear: when an app moves to the background, reclaim its anonymous pages first and preserve file pages. File pages account for the bulk of MemAvailable. Keeping them around means the system reports more “available” memory. More importantly, when a sudden memory demand arises, e.g., the camera launching, clean file pages can be discarded instantly, without waiting for I/O. This is a space-for-latency trade-off, and

memcg 69/apps/X Reclaim from oldest generation.
Node 0 Anonymous page unreclaimable.

Gen.	Birth Time	Anon. Page	File Page
0	114,052	0	0
1	114,052	0	17,728
2	114,052	0	0
3	114,052	148,841	39,050

Figure 2: Page distribution across generations for TikTok.

it works well. The balance between anonymous and file reclaim is therefore critical. MGLRU’s generational mechanism disrupts this balance, as we show in the next section.

3 Problem

In practice, we observe a systematic generational imbalance under MGLRU on Android workloads. Anonymous pages tend to concentrate in the youngest generation, while file pages are scattered across older generations. Because reclaim only operates on the oldest generation, this imbalance causes the kernel to evict file pages much earlier than anonymous pages. The result is a reclaim behavior opposite to Android’s preferred policy, which prioritizes reclaiming anonymous memory while preserving file cache. Although first observed on Android devices, the underlying mechanism is not Android-specific and can affect any workload that prefers preserving file pages over anonymous memory. We first illustrate this phenomenon using a real application example and then analyze the kernel mechanisms that produce it.

3.1 Generational Imbalance

To understand the generational state produced by MGLRU, we inspected the runtime distribution of pages in a typical Android application. Figure 2 shows the `lru_gen` debugfs output for TikTok (`com.ss.android.ugc.aweme`).

The distribution is highly skewed. All 148,841 anonymous pages sit in gen 3, the youngest generation. Meanwhile, 17,728 file pages occupy gen 1. Generations 0 and 2 contain almost no pages. This layout has direct implications for reclaim. Because eviction starts from the oldest generation, file pages in gen 1 become immediate reclaim targets, while anonymous pages concentrated in the youngest generation remain largely untouched. As a result, the system preferentially evicts file pages despite a large amount of resident anonymous memory.

We observed similar generational patterns across multiple popular Android applications, indicating that this behavior is a systematic outcome of the MGLRU placement rules rather than an application-specific artifact.

3.2 Root Cause Analysis

In MGLRU, a page’s target generation is decided by `lru_gen_add_folio()` at `mm_inline.h:242`. The

core logic is as follows:

```
if (folio_test_active(folio))
    seq = lru_gen->max_seq;
else if (type == ANON && !swapcache)
    seq = lru_gen->max_seq - 1;
else if (reclaiming || min_seq + 2 >= max_seq)
    seq = lru_gen->min_seq;
else
    seq = lru_gen->min_seq + 1;
```

On Android, apps generate many anonymous pages at startup. These pages are allocated through the page fault path. During a page fault, the `in_lru_fault` flag is set to true, which causes `folio_add_lru()` to mark the page as active. All anonymous pages, therefore, hit the first branch and go directly into the youngest generation.

File pages take a different path. The Android Runtime (ART) loads DEX and OAT files through the `read()` system call, not through mmap page faults. In the `read()` path, `in_lru_fault` is false. Pages are not marked active. They fall through to the last branch and enter gen 1.

This asymmetry between the page-fault path and the `read()` path creates a systematic placement bias between anonymous and file pages. The mechanism is not Android-specific: any workload combining page-fault-driven anonymous allocations with `read()`-driven file I/O can experience the same imbalance, leading to premature eviction of file pages.

3.3 Why Swappiness Fails

As discussed above, Android prefers to reclaim anonymous pages from background apps while preserving file pages. A natural idea is to raise swappiness (0–200, larger values favor reclaiming anonymous pages over file pages), so the kernel more aggressively reclaims anonymous memory. Unfortunately, the coupling mechanism makes this almost useless.

The type selection logic in `isolate_folios()` (`vmscan.c:5319`) does respect swappiness when choosing whether to scan anonymous or file pages. With `swappiness=200`, the system will indeed prioritize scanning anonymous pages. But the real problem lies in the aging gate.

`should_run_aging()` checks the condition `min_seq[!can_swap] + MIN_NR_GENS > max_seq`. When swap zone is configured and `swappiness > 0`, `can_swap` is true, so the check uses `min_seq[ANON]`. Since anonymous pages only occupy a single generation, `min_seq[ANON]` equals `max_seq`. The condition `min_seq[ANON] + MIN_NR_GENS > max_seq` is therefore always true. This forces aging to run before any reclaim can begin. By default, the system must complete two rounds of page table scanning before it can start evicting. Anonymous pages remain concentrated in the youngest generation.

In short, coupling makes swappiness far less effective at promoting anonymous page reclaim than it is under the traditional LRU. Combined with Android’s preference for preserving file pages, the impact is amplified: file pages are over-reclaimed, anonymous pages remain stuck, and MemAvailable suffers.

4 Mitigations and Large-Scale Evaluation

4.1 Userspace memcg Aging

Our mitigation is straightforward: when an Android app transitions to the background, we trigger aging on its memcg. This forces `max_seq` forward, causing anonymous pages that were concentrated in the youngest generation to start settling into older generations. Once there, they become eligible for reclaim.

The implementation uses an Android vendor hook. When the system detects an app transitioning from the foreground to the background, it writes an aging command to the `lru_gen` debugfs interface. Figure 3 shows the generational distribution before and after two rounds of aging.

After two rounds of aging, anonymous pages in gen 3 dropped from 148,841 to 119,415. About 33,000 anonymous pages were redistributed into gen 4 and gen 5. These pages are now within reach of the kernel.

This approach has limitations. Android app freezing clears the memcg generation state. When an app is thawed, all its pages re-concentrate in the youngest generation. Aging must therefore be triggered repeatedly. Also, the current implementation relies on the debugfs interface, which is not ideal for production use. The kernel community has comments about migrating the control interface to sysfs or BPF [2].

4.2 Large-Scale Field Data

We ran an A/B test on 8GB commercial devices, covering hundreds of users over tens of thousands of device-hours. Table 1 shows the key application performance metrics, measured as event counts per 1,000 hours. Table 2 summarizes system-level metrics.

Table 1: Application performance (events / 1,000 hours).

Metric	Base ver.	Opt. ver.	Impr.
	303 users 45,836h	271 users 29,846h	
Hot start > 450 ms	830.9	716.4	16%
Cold start > 2 s	201.1	185.6	8%
Warm start > 2 s	136.4	121.7	12%
Launch response > 245 ms	38.6	30.2	28%
IME popup > 250 ms	476.6	328.0	45%
IME dismiss > 250 ms	58.0	30.6	89%
Fingerprint unlock > 1 s	1.14	0.94	21%

Table 2: System-level metrics.

Metric	Baseline	Impr.	Change
Avg. app keep-alive	13.2	15.2	+15%
Avg. MemAvailable (MB)	2,257	2,348	+4%
Avg. launch latency (ms)	493.2	483.4	+2%
Reclaim thread power (mAh)	65.74	72.66	−10.5%

The reclaim thread power increased from 65.74 to 72.66 mAh (+10.5%). This is expected. The mitigation increases anonymous page reclaim, which requires zram compression and incurs additional CPU cost. It is a deliberate trade-off: more power for more available memory and better app retention.

Overall, the data confirms that the mitigation works at a commercial scale. App keep-alive rose from 13.2 to 15.2, meaning users can keep more apps in the background. Hot start and IME latency improvements translate directly to a more responsive experience. The MemAvailable gain of 91 MB may seem modest, but on an 8 GB device with only about 2.3 GB actually available, it is meaningful.

4.3 Server Validation

To evaluate whether the imbalance is specific to Android workloads or a general property of MGLRU, we ran a community-provided server benchmark suite on a Linux PC equipped with an Intel Core i7-14700F CPU (20 cores / 28 threads), 32 GB DDR5-5600 memory, and a 1 TB NVMe SSD. The machine runs Ubuntu 24.04.2 LTS with a custom Linux 6.12.0 kernel. The mitigation differs from the Android approach: a kworker thread scans the current MGLRU generation; when anonymous pages are concentrated in the youngest two generations, it triggers one round of aging to advance the generation clock. We tested two workloads: `fio` (file-I/O-intensive) and `memtier-benchmark` (key-value, anon-heavy). Specifically, the `fio` process randomly reads a 48 GB file using the Pareto random distribution, thereby continuously generating file-page pressure. For the key-value workload, we start an independent in-memory `memcached` instance and initialize it with 6,000,000 key-value pairs. All keys are inserted during initialization, while only keys 1–600,000 are repeatedly accessed during runtime. This configuration keeps roughly 10% of the anonymous pages hot, while the remaining pages stay cold.

The results align with our analysis. On `fio`, proactive aging improved read bandwidth by 36.1% and reduced p90 read latency by 82.6% (Table 3). File pages benefited directly from the more balanced generational distribution. On `memtier-benchmark`, the impact was negligible (p50 lat changed by −1.59%), as this workload is dominated by anonymous pages that are less sensitive to file page preservation. These results confirm that the generational imbalance is inherent to MGLRU’s coupled design, not an artifact of the Android platform.

Table 3: Server benchmark results with proactive aging. ↑: higher is better. ↓: lower is better.

Metric	MGLRU	+Aging	Impr.
<code>fio</code> read BW ↑ (MiB/s)	3,982.19	5,418.41	+36.1%
<code>fio</code> read IOPS ↑	1,019,460	1,387,110	+36.1%
<code>fio</code> p90 clat ↓ (ns)	2,618.9	455	+82.6%
<code>memtier</code> p50 lat ↓ (ms)	0.135	0.138	−1.59%
<code>memtier</code> p99 lat ↓ (ms)	0.226	0.241	−6.5%

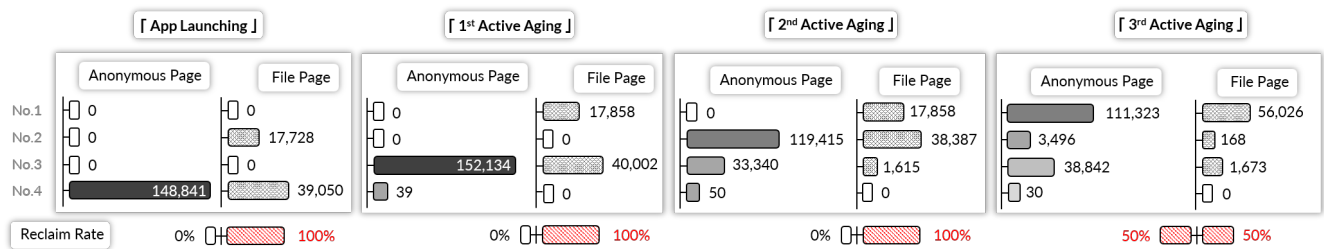


Figure 3: Generational distribution before and after memcg aging.

5 Conclusion

MGLRU’s coupled generation design causes a systematic anon/file imbalance that affects any workload preferring file page preservation, not only Android. We traced the root cause to the page placement rules and the aging gate mechanism, and validated our findings on both mobile and server platforms. Proactive aging mitigates the problem effectively at a commercial scale. The kernel community is discussing decoupled generations for anonymous and file pages [3, 4, 5]. If adopted, this would address the problem at its root. We also recommend migrating the MGLRU control interface from debugfs to sysfs or BPF to support safer and more flexible production control.

References

- [1] Y. Zhao, “Multi-Gen LRU: The Next Generation,” *LWN.net*, 2022. <https://lwn.net/Articles/856931/>
- [2] Z. Wang, “Add memcg aging interface for Android,” *linux-mm mailing list*, Nov. 2025. <https://lore.kernel.org/linux-mm/20251128025315.3520689-1-wangzicheng@honor.com/>
- [3] B. Song, “Re: MGLRU on Android: Real-World Problems and Challenges,” *linux-mm mailing list*, Feb. 2026.
- [4] Z. Wang, “MGLRU on Android: Real-World Problems and Challenges,” *LSF/MM/BPF 2026 topic proposal*, Feb. 2026. https://lore.kernel.org/linux-mm/CAGsJ_4zatnuLkCJyDe_o_yXmhngpc54kV6b7M3tu6JyvF-ZxDw@mail.gmail.com/
- [5] Y. Xie, “Re: MGLRU on Android: Real-World Problems and Challenges,” *linux-mm mailing list*, Feb. 2026.
- [6] S. Baghdasaryan, “Re: MGLRU on Android: Real-World Problems and Challenges,” *linux-mm mailing list*, Feb. 2026.